

Characteristics of data types

When you declare a variable, you're not just naming a box — you're signing a contract that fixes three things at once.

What this key knowledge covers

This summary distils everything you need for **KK02 — Characteristics of data types** in Programming. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

A data type is a contract, not a label

When you declare a variable, you're not just naming a box — you're signing a contract that fixes three things at once.

The set of values the variable is allowed to hold. A Boolean can only ever be true or false. An integer can't hold 3.5. Try and the language either rejects it or quietly mangles it.

A Boolean needs about a bit; an integer a few bytes; a long string grows with its length. The type tells the computer how much room to reserve.

You can do arithmetic on numbers but not on text. You can join (concatenate) strings but not Booleans. The `+` symbol even *means different things* depending on the type — that's the trap that catches the most students.

Think of a school canteen form. The **“how many?”** box only accepts whole numbers — you can't order 2.5 sausage rolls. The **“name”** box accepts any text. The **“gluten free?”** box is a tick or no-tick — nothing in between. Nobody wrote rules on the form for fun: each box's *type* stops nonsense getting in and tells the canteen what it can do with each answer. Data types are exactly that, enforced by the computer instead of a canteen lady.

The data type you choose decides what operations the computer will let you perform on that value. Almost every exam mark on this dot point comes back to that idea.

The Type Foundry

Pick a real Lockr value below. Watch it drop into its matching container, then read what that type can hold, how much memory it needs, and which operations it unlocks. Try the 0042 chip — it hides a classic trap.

Each chip is a field Priya must store in Lockr. Choosing a value reveals its data type and everything that follows from it.

Text types

Text data is anything made of letters, digits-as-symbols, spaces and punctuation. The study design splits it into two: **character** and **string**.

Exactly **one** symbol — a single letter, digit, space or punctuation mark. In Lockr the locker block is one of A – F, so Priya stores block as a character.

Holds: 'C', '7', '?', ' ' · **Quoted:** usually single quotes

A **sequence** of characters of any length, including empty. A student's name, an email, a 4-digit PIN — all strings. "Priya Nair" is one string of 10 characters.

Holds: "", "C", "Block C – Level 2" · **Quoted:** usually double quotes

A phone number, a postcode, a PIN and a student ID all *look* like numbers but should be **strings**. Why? You never do arithmetic on them, they can start with 0 (which an integer throws away — 0042 becomes 42), and they can be too big for a numeric type. **Rule of thumb: if you'd never add or average it, it's text.**

A **character** is a single Scrabble tile. A **string** is the whole word you build by laying tiles in a row. A word can be one tile long, or twenty, or zero (an empty rack). That's the only real difference: a character is one tile; a string is the row.

Numeric types

Numeric data is data you genuinely do maths on. The study design lists three numeric forms, and choosing between them is a favourite exam question.

Whole numbers, positive or negative, no decimal point. Counting things. Lockr stores lockerNo = 47 and lockersLeft = 12 as integers.

Numbers with a decimal/fractional part. Money, weights, percentages, sensor readings. Lockr stores dailyFee = 1.25 as a floating-point number.

Holds: 1.25, -0.5, 3.14159, 99.0

A **specialised numeric type** for points in time. It knows that the day after 31 Dec is 1 Jan, and it can be subtracted to find durations. Lockr stores bookedUntil = 12 Sep 2026.

Holds: 12/09/2026, 08:45, full timestamps

Ask: **“Can this value ever have a fraction?”** A count of lockers can't (you never have 12.5 lockers) → **integer**. A fee, a temperature or a weight can → **floating point**. Choosing integer for money silently rounds \$1.25 down to \$1 and Priya loses 25c on every booking.

An **integer** is a staircase — you can only stand on whole steps, 1, 2, 3. A **floating point** is a ramp — you can stop anywhere, including 2.37 of the way up. Counting people? Stairs. Measuring how full the water tank is? Ramp.

Boolean — the decision type

A Boolean holds one of exactly two values: true or false. It's the smallest, simplest type — and the one your program's logic runs on. Named after mathematician George Boole.

Lockr uses Booleans for every yes/no fact: isPaid, isOverdue, isMember. These feed straight into if statements — the value of a Boolean decides which branch the program takes. Flip the gate below and watch the path change.

Because branching is the heart of programming. `if (isPaid) openLocker() else showPayment()` reads almost like English. Storing “paid?” as the string “yes” instead would force messy comparisons and let typos like “Yes” or “y” break the logic. A Boolean makes the decision crisp and impossible to misspell.

The same symbol, different jobs

This is the single most-tested misconception on this dot point. The `+` operator does **addition** on numbers but **concatenation** (gluing end-to-end) on strings. Same key on your keyboard, completely different result.

Priya reads a quantity from a text box and writes `total = qty + 1`. Text boxes hand back **strings**, so if a user types “2”, she gets “21”, not 3. The fix is to **convert (cast)** the string to an integer first. No error appears — the program just silently does the wrong thing. That’s why understanding types prevents bugs you’d otherwise never spot.

Picking well — Priya’s data dictionary

In the SAC you’ll be asked to **state and justify** the most appropriate type for a field. Run every field through two questions: “*Will I ever do maths on it?*” and “*Can it have a fraction or a leading zero?*” Tap a card to reveal Priya’s decision and her reasoning.

1. Yes/no fact? → **Boolean**. **2.** A point in time? → **date/time**. **3.** Will you do arithmetic on it? If no → **text** (string, or character if always one symbol). **4.** If yes, can it have a decimal? Yes → **floating point**; No → **integer**.

Sort it — drag each value to its type

Drag (or tap on touch) each Lockr value into the correct type bin, then hit **Check**. Wrong ones go red so you can rethink them.

Six ways students lose marks here

Every one of these is something examiners’ reports flag year after year. Open each, learn the fix.

Write answers that earn the marks

Half the battle is reading the command word and answering in its shape. Then structure justifications with **P•E•L** — Point, Example, Link.

Six fast questions

Instant feedback. These are the “do you actually know it” reps — no marks recorded, just self-test.

VCE-style practice — mark your own

Five questions, **16 marks** total, written in VCAA’s voice. Attempt each on paper first, then reveal the weak answer, the strong answer and the marking guide. Award yourself marks — your score follows you in the dock.

- “Number” isn’t specific — the study design wants integer vs floating point. “Yes/no” isn’t a data type.

- Names the right type but gives no real problems — “numbers are bad” explains nothing. Only 1 mark.
- Vague and missing an example for floating point. No clear contrast of the fractional part. ~1 mark.
- (a) is wrong — an integer drops the leading zero. (b) names the type but the justification is thin. Roughly 1–2 marks.
- No mention of data types at all — the question's core. Doesn't identify string concatenation. ~0–1 marks.

Cheat sheet

The whole dot point on one screen. Screenshot it the night before.

Type = a contract fixing what fits, how much memory, and which operations are legal. · **Digits you never add are text.** · **Fraction possible → float, else integer.** · **+ adds numbers but glues strings.**

Common traps & misconceptions

Exam trap

T1 Calling a phone number / PIN “a number” ► They contain digits, but you never do arithmetic on them and they can start with 0. Stored as an integer, 0042 becomes 42 and the value breaks. **Fix** ✓ Store IDs, phone numbers, postcodes and PINs as **strings**. Rule: if you'd never add or average it, it's text.

Exam trap

T2 Using an integer for money ► Choosing integer for a price silently throws away the cents — \$1.25 is stored as 1. Across many bookings the totals are wrong and no error appears. **Fix** ✓ Money needs a fractional part, so use **floating point**.

Exam trap

T3 Thinking + always adds ► On strings, + **concatenates**. "5" + "3" gives "53", not 8. Input from a text box is a string, so maths on it misbehaves. **Fix** ✓ Know the operand types. **Convert (cast)** text to a number before doing arithmetic.

Exam trap

T4 Confusing character and string ► A **character** is one symbol; a **string** is a sequence. Writing “character” when the field is a whole name (or vice-versa) loses the mark. **Fix** ✓ One symbol only → character. Any length, including one → string.

Exam trap

T5 Storing yes/no as text ► Using the string "yes" for a paid flag invites typos ("Yes", "y") and clunky comparisons that break the logic. **Fix** ✓ Use a **Boolean** — it's exactly two values and feeds straight into an if.

Exam trap

T6 Not justifying, just naming ► A **justify** question wants reasons. Answering only “string” for a 2-mark justify gets 1 mark at best. **Fix** ✓ State the type **and** give a reason tied to the field — ideally why the alternative fails.

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. Justify · 2 marks

A program stores a student's phone number. Justify whether it should be declared as an **integer** or a **string**.

✗ A weak answer

It should be an integer because a phone number is numbers. (→ *Surface reasoning; ignores leading zeros and that no arithmetic is done — 0-1.*)

✓ A strong answer

It should be a **string**. Although a phone number looks numeric, it is never used in calculations, it can begin with a leading zero (which an integer would drop), and it may contain spaces or a "+" country code. A string preserves every character exactly, so string is the correct data type.

How the marks are awarded

- 1 mark — selects string.
- 1 mark — justifies with a valid reason (leading zeros / no arithmetic / formatting characters).

Q2. Define · 2 marks

Define the **Boolean** data type and give one example of where it would be the most appropriate choice in a program.

✗ A weak answer

Boolean is true or false. (→ *Definition is fine but no example, so it caps at 1 mark.*)

✓ A strong answer

A **Boolean** is a data type that can hold only one of two values: **true** or **false**. It is the most appropriate choice for storing a yes/no state — for example, an *isPaid* flag that records whether a canteen order has been paid for.

How the marks are awarded

- 1 mark — defines Boolean as a two-value (true/false) type.
- 1 mark — gives a valid yes/no example use.